Course Outline: Sending Telemetry from the Frontend

Title: Sending Telemetry from the Frontend **Skill:** 43 — Collecting telemetry and context-related info from the user and your application **Learnpack:** + Enviando telemetría desde el frontend de mi aplicación **File:** s43-w15d43-enviando-telemetria-desde-el-frontend-de **Prerequisite:** Understanding Telemetry from the Frontend (Skill 43 — what to collect and why) **Target Audience:** Beginner developers in a full-stack AI bootcamp who know what signals to collect and are ready to implement the sender **Total Lessons:** 10 **Format:** Mixed — 6 📖 + 1 💻 + 1 🧠 + 1 outro (10% hands-on)

Course Description

This course bridges the gap between knowing what to collect (previous learnpack) and actually sending it. Students design the JSON event schema, implement a telemetry sender that batches events and flushes them reliably, and internalize a 6-step mental model for making sound implementation decisions. By the end, they have a working telemetry client in JavaScript.

Course Philosophy

This course emphasizes:

- Contract before code: the event schema is a specification, not an afterthought
- Reliability as a first-class concern: telemetry that drops events under load or on page unload is broken telemetry
- One mental model over many patterns: the 6-step checklist unifies all implementation decisions

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Event Contract	2
02 - Sending Telemetry	4
03 - Assessment	2
04 - Outro	1
Total	**10**

00.0 Welcome 📖

Learning Objectives:

- Understand what this course covers: designing the event contract and implementing a reliable telemetry sender
- Connect this course to the previous (taxonomy of what to collect) and the next (Next.js-specific capture hooks)
- Understand why the event schema must be designed before the sender is built

Content Outline:

- The gap this course fills: knowing what to collect is not the same as knowing how to format and send it
- Two phases: first, the event contract (what each event looks like as JSON); second, the sender (how to transmit events reliably)
- Preview: the 6-step mental model that connects every decision in this course

Transitions: In the previous course, you learned the taxonomy: identity, profile, execution context, product usage events, performance, quality, and consent. This course answers the next question: how do you turn those signals into JSON events and reliably get them to your backend?

Key Principles:

- The event contract is a team agreement — both frontend and backend must agree on the schema before either side builds
- Reliability is not optional — a telemetry system that silently drops events under any condition is not a telemetry system

Examples:

- Spotify's recommendation algorithm degrades immediately if event transmission fails — their teams invest heavily in sender reliability for exactly this reason
- A batch-and-debounce sender reduces HTTP requests by 95% compared to sending each event individually, without losing any data

---

01.0 The Event Contract 📖

Learning Objectives:

- Understand what an "event contract" is and why it must be established before implementation
- Preview the four components of a well-designed event: base fields, typed properties, PII minimization, and schema stability
- Understand the data-to-decision principle for deciding what to include in an event

Content Outline:

- What an event contract is: the agreed-upon JSON shape for every telemetry event
- Why it must exist before implementation: if the schema changes frequently, all downstream consumers (analytics dashboards, ML models, reporting pipelines) break
- The four contract components:
    1. Base fields: the envelope present in every event (event name, timestamp, identity, session, context, properties)

2. Typed and stable properties: specific, named fields — not open-ended objects
3. PII minimization: no raw emails, phone numbers, or names in telemetry
4. Schema versioning: how to evolve the contract without breaking consumers

- The data-to-decision principle: "What decision does this field enable?" — if no one can answer, the field doesn't belong

Transitions: The event contract is the specification. The next lesson designs the JSON schema in detail.

Key Principles:

- An event contract is not a database schema — it evolves, but changes must be coordinated across frontend, backend, and analytics
- Unstable property names are one of the most common sources of broken dashboards and ML model degradation
- "Add more fields later" is the enemy of clean telemetry — start with the minimum that answers a specific question

Examples:

- Stripe's events API uses a fixed base schema that has barely changed in 10 years, making it trivially easy to build on top of
- A `properties: {}` object with no typed fields is tempting but means every query requires schema documentation to interpret

---

01.1 Designing the Event Schema 🕮

Learning Objectives:

- Design the base fields of a telemetry event as a concrete JSON object
- Apply PII minimization rules to a telemetry event
- Explain the difference between `userId` and `anonymousId` as identity fields in the event envelope

Content Outline:

- The base event object structure:

```
{
  "event": "checkout_submitted",
  "timestamp": "2026-04-24T10:30:00.000Z",
  "user": {
    "userId": "usr_abc123",
    "anonymousId": null,
    "sessionId": "sess_xyz789"
  },
  "context": {
    "appVersion": "2.4.1",
    "env": "production",
    "route": "/checkout",
    "featureFlags": { "new_checkout_flow": true }
```

```
    },
    "properties": {
      "checkout_total": 149.99,
      "payment_method": "card",
      "item_count": 3
    }
  }
```

- Field-by-field explanation:
  - `event`: snake_case verb-noun naming convention (e.g., `checkout_submitted`, `course_enrolled`, `search_performed`)
  - `timestamp`: ISO 8601, generated at event creation, not at send time (batch delay can shift send time)
  - `user.userId`: present when authenticated, null otherwise
  - `user.anonymousId`: present when unauthenticated, null once authenticated (identity stitching handled separately)
  - `user.sessionId`: always present
  - `context`: execution context fields (from the previous learnpack — appVersion, env, route, featureFlags)
  - `properties`: event-specific typed fields — no open objects, no PII
- PII minimization rules:
  - Never include: email addresses, phone numbers, full names, street addresses in `properties`
  - Use opaque IDs: `userId: "usr_abc123"` not `email: "user@example.com"`
  - Hash sensitive identifiers at the source if needed for matching
- Stable properties: agree on names and types upfront — `checkout_total` (number) not `total` (string sometimes, number other times)

Transitions: You now have the JSON shape. The next section implements the code that batches these events and transmits them to the backend.

Key Principles:

- Timestamp at event creation, not at send time — a batch delayed by 5 seconds should not corrupt the event timeline
- `anonymousId` and `userId` are mutually exclusive in the event (one is null) — the backend resolves identity stitching offline
- Properties that change schema (type or name) break analytics retroactively — versioning or coordination is required

Examples:

- `event: "click"` is a broken event name — what was clicked? Use `event: "add_to_cart_clicked"` or, better, `event: "cart_item_added"`
- `properties: { email: "user@example.com" }` is a GDPR/CCPA violation — use `properties: { userId: "usr_abc123" }` and resolve email server-side

---

02.0 Sending Telemetry to the Backend 🕮

Learning Objectives:

- Understand the three core transmission decisions: endpoint design, batching strategy, and authentication
- Preview the reliability patterns that make a production telemetry sender robust
- Explain why individual-event-per-request sending is impractical at scale

Content Outline:

- The transmission problem: millions of events per day, from thousands of concurrent users, must reach the backend without overwhelming it or the client
- The three transmission decisions:
    1. Endpoint: POST /telemetry/events — single endpoint, always accepts arrays
    2. Batching strategy: batch + debounce (accumulate events for N milliseconds OR until N events, then flush)
    3. Authentication: token or cookie in the request header; for unauthenticated users, `anonymousId` identifies the session
- Why individual-event-per-request fails:
    - A single page load generates 10-50+ events
    - 1,000 concurrent users × 50 events × 1 request each = 50,000 requests/minute to your telemetry endpoint
    - With batching: same 50,000 events in ~1,000 requests — a 98% reduction
- Preview of reliability patterns: sendBeacon for unload flush, local retry queue for network failures, correlation IDs for tracing

Transitions: The transmission decisions are set. The next lesson implements the batch sender, and the one after adds reliability.

Key Principles:

- POST /telemetry/events always accepts an array — even if it contains one event — so the client never needs to know whether it's sending 1 or 100
- Debounce is the mechanism that makes batching work in real time: don't wait for 100 events, flush after X milliseconds of accumulation
- Authentication should be transparent to the telemetry client — it reads the token from your existing auth state, not from a separate login flow

Examples:

- Amplitude's SDK batches events internally, which is why you can call `track("event")` 50 times in a second without generating 50 HTTP requests
- A news site that sends individual events for every scroll position update would make its telemetry backend unresponsive within seconds of a traffic spike

---

02.1 The Batch Sender 🕮

Learning Objectives:

- Explain how batch + debounce works as an event accumulation and flush strategy
- Describe what fields must be included in the POST /telemetry/events request (headers, body format)
- Explain how correlation IDs connect client-side events to backend logs

Content Outline:

- The batch + debounce algorithm:
    1. Event arrives → push to in-memory buffer
    2. Start (or reset) a debounce timer (e.g., 3 seconds)
    3. When timer fires → flush: POST all buffered events to /telemetry/events, clear buffer
    4. If buffer reaches N events (e.g., 50) → flush immediately without waiting for timer
    5. On page visibility change (hidden) → force-flush
- The POST request:
    - URL: `POST /telemetry/events`
    - Headers: `Authorization: Bearer <token>` (or cookie), `Content-Type: application/json`
    - Body: `{ "events": [ ...event objects... ] }`
- Correlation IDs:
    - If the backend returns a `X-Request-ID` or `X-Trace-ID` header in responses, include it in subsequent events as `context.requestId`
    - Enables end-to-end tracing: frontend event → API call → backend log → telemetry event
- Anonymous user authentication: `anonymousId` in the event payload identifies the session — no auth token required, the backend trusts the ID
- Request failure handling (brief): what happens when the POST fails — covered in the next lesson

Transitions: The batch sender works for normal conditions. The next lesson adds the reliability patterns that handle edge cases: page unloads and network failures.

Key Principles:

- "Debounce" (reset timer on each new event) is correct here, not "throttle" (fire every N milliseconds) — debounce waits for a quiet moment, throttle fires on a fixed schedule
- The buffer should have a hard cap (e.g., 100 events) to prevent memory growth during network outages
- Force-flushing on `visibilitychange` is the last chance to send buffered events before the user closes the tab

Examples:

- A user who triggers 20 events in 2 seconds will have them all sent in a single POST after the debounce fires — not 20 separate requests
- A checkout page where the user completes the purchase then immediately closes the tab will lose the `checkout_completed` event if there's no force-flush on `visibilitychange`

02.2 Reliability Patterns 🕮

Learning Objectives:

- Explain how navigator.sendBeacon enables telemetry flushing on page unload
- Describe a local retry queue with exponential backoff for handling network failures
- Understand when to use sendBeacon vs fetch

Content Outline:

- The unload problem: when a user closes a tab, in-flight `fetch` requests are cancelled by the browser — events in the buffer are lost
- `navigator.sendBeacon`: the browser API designed specifically for unload-time telemetry
  - Sends a small payload in the background even as the page is being destroyed
  - Guaranteed delivery for short payloads (< 64KB)
  - No response is received — fire and forget
  - Used in: `visibilitychange` (document.visibilityState === 'hidden') + `pagehide` handlers
  - Format: `navigator.sendBeacon('/telemetry/events', JSON.stringify({ events: buffer }))`
- Local retry queue:
  - When a POST /telemetry/events request fails (network error, 5xx), events should not be dropped
  - Store failed events in `localStorage` (persistent across page reloads) or in-memory queue (lost on reload)
  - Retry with exponential backoff: wait 1s, 2s, 4s, 8s... up to a max retry count (e.g., 3)
  - On retry success: clear the queue; on max retries exceeded: discard (events are not critical data)
- When to use what:
  - Normal operation: `fetch` with batch + debounce
  - Page unload: `navigator.sendBeacon`
  - Network recovery: retry from local queue

Transitions: You now have the complete design. The next lesson puts it into code.

Key Principles:

- `sendBeacon` only works for small payloads — don't accumulate thousands of events expecting a reliable unload flush; flush frequently
- Telemetry events are not transactions — losing some under extreme conditions (max retries exceeded, 100MB buffer cap) is acceptable; losing all events regularly is not
- In-memory queues are lost on page reload; `localStorage` queues survive — choose based on your reliability requirements

Examples:

- Google Analytics uses `sendBeacon` for its final page-exit hits (session duration, bounce rate) — without it, ~15% of sessions would be inaccurate
- A retry queue that retries infinitely can cause a "retry storm" when a backend comes back online after outage — exponential backoff prevents this

---

02.3 Build the Telemetry Sender 💻

Challenge Prompt: Build a JavaScript telemetry sender module that batches events and sends them reliably. The module should expose a `track(eventName, properties)` function that buffers events and flushes them to `POST /telemetry/events` after 3 seconds of inactivity or when 20 events accumulate. On page hide, flush immediately using `navigator.sendBeacon`.

Solution Code:

```
const ENDPOINT = '/telemetry/events';
const DEBOUNCE_MS = 3000;
const MAX_BATCH = 20;

let buffer = [];
let timer = null;

function getBaseContext() {
  return {
    appVersion: window.__APP_VERSION__ || 'unknown',
    env: process.env.NODE_ENV || 'production',
    route: window.location.pathname,
  };
}

function flush() {
  if (buffer.length === 0) return;
  const events = [...buffer];
  buffer = [];
  clearTimeout(timer);
  timer = null;
  navigator.sendBeacon(ENDPOINT, JSON.stringify({ events }));
}

function flushWithFetch() {
  if (buffer.length === 0) return;
  const events = [...buffer];
  buffer = [];
  clearTimeout(timer);
  timer = null;
  fetch(ENDPOINT, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ events }),
    keepalive: true,
  }).catch(() => {});
}

export function track(eventName, properties = {}) {
  buffer.push({
    event: eventName,
    timestamp: new Date().toISOString(),
```

```
    context: getBaseContext(),
    properties,
  });
  if (buffer.length >= MAX_BATCH) {
    flushWithFetch();
    return;
  }
  clearTimeout(timer);
  timer = setTimeout(flushWithFetch, DEBOUNCE_MS);
}

document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'hidden') flush();
});
```

Challenge Code:

```
const ENDPOINT = '/telemetry/events';
const DEBOUNCE_MS = 3000;
const MAX_BATCH = 20;

let buffer = [];
let timer = null;

function getBaseContext() {
  // Return an object with appVersion, env, and route
  // your code here
}

function flush() {
  // Use navigator.sendBeacon to send buffered events
  // Clear buffer and timer
  // your code here
}

function flushWithFetch() {
  // Use fetch (POST) to send buffered events to ENDPOINT
  // Clear buffer and timer
  // your code here
}

export function track(eventName, properties = {}) {
  // Add event to buffer with timestamp, context, and properties
  // If buffer reaches MAX_BATCH, call flushWithFetch immediately
  // Otherwise reset debounce timer
  // your code here
}

// Set up visibility change listener for page unload flush
// your code here
```

03.0 The Implementation Mental Model 🕮

Learning Objectives:

- Apply the 6-step checklist to a new telemetry implementation decision
- Identify which step in the checklist each pattern from this course addresses
- Use the checklist to audit an existing telemetry implementation for gaps

Content Outline:

- The 6-step implementation mental model:
    1. **Question** → **Data**: "What decision does this telemetry enable?" — if no one can answer, don't collect it
    2. **Data** → **Source**: "Does this data come from the user, the browser, or the application state?" — determines where to hook
    3. **Source** → **Risk**: "Can this source produce PII or secrets accidentally?" — emails, session tokens, raw user input
    4. **Risk** → **Control**: "What guardrails apply?" — allowlist of property keys, consent check, scrub free-form text, sampling rate
    5. **Control** → **Contract**: "What is the stable JSON shape for this event?" — base fields + typed properties
    6. **Contract** → **Transport**: "How is this transmitted?" — batch + debounce via fetch, sendBeacon on unload, retry queue for failures
- Mapping this course to the model:
    - Steps 1-2: covered in the previous learnpack (taxonomy — what to collect and why)
    - Steps 3-4: covered in the next learnpack (guardrails — consent, scrubbing, allowlists)
    - Steps 5-6: covered in this course (event contract + sender)
- How to use it for auditing:
    - For each existing event: can you answer all 6 questions?
    - If step 1 has no answer → the event is noise, remove it
    - If step 4 has no controls → the event is a privacy liability
    - If step 6 has no retry mechanism → events are silently dropped under network stress

Transitions: Test your understanding of everything in this course before moving on.

Key Principles:

- The checklist is a forcing function — it prevents building before defining
- Steps 1-4 protect the company (privacy, legal, signal quality); steps 5-6 protect the product (data integrity, reliability)
- A telemetry system is only as good as the weakest step in the chain

Examples:

- A team adding `user.email` to every event violates step 3 (source risk) and step 4 (no controls) — the checklist catches this before production

- A team using individual fetch per event violates step 6 (transport) — the checklist surfaces this before the backend gets overwhelmed

---

03.1 Telemetry in Transit 🫣

Learning Objectives:

- Apply event schema design rules to a new event
- Identify reliability failures in a described implementation
- Map implementation decisions to the 6-step mental model

Question Format: Scenario-based

Questions:

1. A developer logs the following telemetry event: `{ event: "click", properties: { email: "alice@example.com", button: "Buy Now" } }`. Identify all problems with this event and rewrite it correctly following the event schema rules from this course.

2. A telemetry sender sends each event immediately as a separate `fetch` POST. The app has 1,000 daily active users and each generates 30 events per session. How many requests does this create per day? What would batching reduce this to, assuming an average batch size of 15?

3. A user completes a checkout then immediately closes the browser tab. Your sender uses `fetch` with a 3-second debounce. Will the `checkout_submitted` event be delivered? Why or why not? What should you change?

4. Your telemetry backend returns a `X-Trace-ID: abc123` header for every request. Your team wants to correlate frontend events with backend logs. Where in the event schema does this ID belong, and how do you obtain it in the frontend sender code?

5. Apply the 6-step mental model to the following scenario: "We want to track how long users spend reading each article on our news site." Walk through all 6 steps and describe what your implementation would look like at each step.

Evaluation Criteria:

- Q1: Does the student catch the vague event name (`click`), the PII violation (`email`), and correct both?
- Q2: 1000 × 30 = 30,000 requests/day without batching; 30,000 / 15 = 2,000 with batching
- Q3: No — the fetch is cancelled on tab close. Fix: use `navigator.sendBeacon` in a `visibilitychange` handler
- Q4: `context.traceId` — obtained from the response header of the last flush: `response.headers.get('X-Trace-ID')`
- Q5: Evaluates application of all 6 steps; correct answers include: Q (know reading duration enables content prioritization), Data (time on page — from browser), Source (no PII risk), Control (sampling at 10% for verbose data), Contract (article_read event with duration_seconds as typed number), Transport (batch + debounce + sendBeacon)

Score Interpretation:

- 4-5 correct: Strong grasp of event schema design, reliability patterns, and the implementation mental model
- 2-3 correct: Good foundation; review the batch sender and reliability patterns
- 0-1 correct: Return to the event contract and sender sections

---

04.0 Outro

Content Outline:

- Course recap: what students accomplished
  - Designed a well-formed JSON event schema with base fields, typed properties, and PII minimization
  - Implemented a batch sender using fetch with debounce and navigator.sendBeacon for page unload reliability
  - Internalized the 6-step mental model that connects every implementation decision
- Connection to bigger picture
  - The sender is now complete: you can format and reliably transmit any telemetry event
  - The next learnpack (Capturar telemetría en Next) focuses on WHERE to hook in Next.js — router events, user actions, error handlers, and Web Vitals — using the sender you built here
  - The learnpack after that (Guardrails) adds the controls from step 4 of the mental model: consent checks, scrubbing, allowlists, and environment separation
- Next steps and continued learning
  - OpenTelemetry JavaScript SDK: for teams that want a standardized, vendor-neutral sender
  - web-vitals npm package: built-in support for reportWebVitals with your batch sender
  - navigator.sendBeacon MDN documentation: browser support tables and payload size limits
- Encouragement
  - Most developers treat telemetry as a nice-to-have until something goes wrong. You now have the vocabulary and tools to build it right from the start.

Note: This is conclusion only — no theory, exercises, or assessment.